

Technical report

Introduction

This is a project to demonstrate atmega328 sleep modes. The main objective here is to implement power saving options so that the devices created will have longer battery life or low power consumption. Bare metal codes will be written configured with the desired sleep mode. Protues will be used as demonstration mechanisms.

System architecture

Items used

ATmega328P microcontroller

8 MHz crystal

2× 22 pF ceramic capacitors (crystal load caps)

100 nF ceramic capacitors (VCC decoupling)

10 kΩ resistors (reset pull-up)

100 nF capacitors (reset network)

Red LED

220 Ω resistors (LED current limiting)

Push button

NPN transistor (2N2222 or BC547)

1 kΩ resistor (transistor base)

Buzzer

16×2 LCD (LM016L) — or Virtual Terminal as an alternative

10 kΩ potentiometers (LCD contrast)

5V power supply / ground rails

Registers used

SMCR (Sleep Mode Control Register)

This register selects and arms the sleep mode. Bits SM2, SM1, and SM0 are set to 0, 1, 0 respectively, which selects Power-down mode.

EIMSK (External Interrupt Mask Register)

Only the INT0 bit is set to 1. INT1 stays at 0 since it's unused. Setting INT0 is what allows the INT0 interrupt request to actually be recognized by the CPU.

EICRA (External Interrupt Control Register A)

All four relevant bits — ISC01, ISC00, ISC11, and ISC10 — are set to 0. ISC01:ISC00 = 00 configures INT0 as a low-level trigger rather than an edge trigger. This is the critical choice: level triggering is the only mode detected asynchronously, so it's the only one that can wake the MCU from Power-down, since edge detection depends on the I/O clock, which stops in that mode.

DDRD (Port D Data Direction Register)

Bit DDD2 is cleared to 0, configuring PD2 as an input so it can read the button state.

PORTD (Port D Data Register)

Bit PORTD2 is set to 1, which enables the internal pull-up resistor on PD2. This makes the pin idle high when the button is released, and it gets pulled low when the button is pressed.

DDRB (Port B Data Direction Register)

Bits DDB0 and DDB1 are both set to 1, configuring PB0 (LED) and PB1 (buzzer driver transistor) as outputs.

DDRC (Port C Data Direction Register)

Bits DDC0 through DDC5 are all set to 1, configuring PC0–PC5 as outputs for the LCD's RS, E, and four data lines (D4–D7).

SREG (Status Register)

The I bit (bit 7, global interrupt enable) is set to 1 via `sei()` before entering sleep. Without this bit set, INT0 could still be requested at the hardware level, but the CPU would neither service the ISR nor actually wake up — global interrupt enable is a prerequisite for both.

Implementation and design

Led red is configured to be output at PB0.

Crystal oscillator is connected at PB6 and PB7

Buzzer is connected to PB1.

Button connected at PD2 or INT0

Lcd 16x2 Data pins connected to PC0-PC5

Flow order

The device is powered on with a message at the lcd “normal op /sleep demo “

Although the code is configured to go to sleep after 4000ms it takes more time in protues

After elapsed time the system goes to power down mode and major peripherals are disabled and the lcd displays sleeping

Note – power down disables peripherals but how is the lcd “displaying sleeping” it is because the moment the system goes to sleep the system sends the message to the lcd module and stored in its ram.

The system waits for external interrupt for the button to be pressed.

When the button is pressed the lcd goes blank for few MS it was because of the phenomenon explained earlier and proceed to running operation again and the loop continuous

Conclusion

There are various sleep modes to be used. the desired one will be different in every scenario, and the proper one must be chosen.